

Good. Now we reach a very important point.

Most engineers jump directly from digital design to ARINC 818 and struggle.

The missing bridge is what I call:

PHASE 0.5

ARINC 818 Prerequisite Domain Knowledge

This phase exists because ARINC 818 is **not a protocol invented from scratch**.

It sits on top of several technologies:

Video Systems



Avionics Display Systems



Fibre Channel



High-Speed Serial Communication



SERDES/GT Transceivers



FPGA Implementation

If Phase 0 taught you how digital systems work, Phase 0.5 teaches you the world ARINC 818 lives in.

PHASE 0.5 OBJECTIVE

By the end you should be able to explain:

- What a video stream actually is.
- How pixels become bandwidth.
- Why aircraft need ARINC 818.
- What Fibre Channel is.
- Why Fibre Channel was chosen.

- What GT transceivers do.
- How high-speed serial links work.
- How video becomes ARINC packets.
- How ARINC packets become serial bits.
- What can go wrong in the real world.

Before opening a single ARINC 818 page.

MODULE 0.5.1

Video Fundamentals

This is the biggest hidden prerequisite.

Many FPGA engineers know RTL.

Few know video.

ARINC 818 transports video.

Without video knowledge, the specification becomes difficult.

What Is A Pixel?

A pixel is:

Smallest displayable picture element.

Example:

1920 × 1080

means:

1920 pixels horizontally

1080 pixels vertically

Total:

2,073,600 pixels

per frame.

Pixel Data

Each pixel contains information.

Example:

RGB.

Red
Green
Blue

Each component may be:

8 bits

Therefore:

24 bits/pixel

Example:

Pure red:

$R = 255$
 $G = 0$
 $B = 0$

Binary representation exists behind this.

Color Depth

Need understanding of:

8-bit color

16-bit color

24-bit color

30-bit color

36-bit color

Why important?

Bandwidth calculations depend on this.

Resolution

Need understanding of:

640×480

800×600

1280×720

1920×1080

2560×1440

3840×2160

Resolution directly impacts:

Bandwidth

Memory

Link Speed

Frame

A frame is:

One complete image.

Example:

60 FPS

means:

60 complete images/sec

Frame Rate

Need understanding:

30 FPS

60 FPS

120 FPS

240 FPS

Higher FPS:

- smoother motion
 - more bandwidth
-

Essential Calculation

Example:

1080p60

24-bit RGB

Bandwidth:

$1920 \times 1080 \times 24 \times 60$

=

2.986 Gbps

raw.

This calculation must become automatic.

MODULE 0.5.2

Video Timing

One of the most misunderstood topics.

Pixels are not transmitted continuously.

There are timing intervals.

Need understanding:

Active Video

Actual image data.

Horizontal Blanking

Gap between lines.

Vertical Blanking

Gap between frames.

Historically needed for CRT.

Still exists.

Video timeline:

Line

└ Active Pixels
└ Horizontal Blank

Frame

└ Active Lines
└ Vertical Blank

Need understanding of:

HSYNC

Horizontal Sync

VSYNC

Vertical Sync

DE

Data Enable

These signals appear in FPGA video systems.

MODULE 0.5.3

Video Data Path

Understand complete flow.

Example:

Camera



Image Sensor



FPGA

↓
Frame Buffer
↓
ARINC 818
↓
Display

Need ability to draw:

Source
↓
Processing
↓
Transport
↓
Sink

for any video system.

MODULE 0.5.4

Aerospace Display Systems

Now move toward ARINC.

Question:

Why was ARINC 818 created?

Need system understanding.

Aircraft contain:

Primary Flight Display

Navigation Display

Engine Display

Mission Display

Radar Display

Sensor Display

These systems exchange video.

Requirements:

High Reliability

Low Latency

Long Distance

EMI Resistance

Determinism

These requirements shaped ARINC 818.

MODULE 0.5.5

Fibre Channel Fundamentals

This is the most important prerequisite.

ARINC 818 is built on Fibre Channel.

Many ARINC concepts come directly from Fibre Channel.

What Is Fibre Channel?

Originally developed for:

High-speed data transport

especially storage systems.

Not Ethernet.

Not PCIe.

Different philosophy.

Need understanding of:

Frame-Based Communication

High Reliability

Deterministic Operation

Fibre Channel Layers

Simplified:

FC-4

FC-3

FC-2

FC-1

FC-0

Do not memorize.

Understand:

Upper layers use lower layers.

FC Frame

Fundamental transport unit.

Contains:

SOF
Header
Payload
CRC
EOF

This becomes critical later.

Why Frames?

Need:

Addressing
Control
Error Detection
Payload Transport

Ordered Sets

Critical concept.

Special symbols used for:

Idle
Link Control
Synchronization

Fibre Channel Link Rates

Historically:

1G
2G
4G
8G
16G
32G

Need awareness.

MODULE 0.5.6

High-Speed Serial Communication

Now we move closer.

Question:

Why not use parallel buses?

At high speeds:

Problems:

Skew

Crosstalk

Routing

EMI

Therefore:

Use serial links.

Example:

4 Gbps serial

instead of:

32-bit parallel bus

Differential Signaling

Fundamental.

Two wires:

TX+

TX-

Receiver observes:

Difference

not absolute voltage.

Advantages:

Noise Immunity

Higher Speed

Lower EMI

MODULE 0.5.7

Encoding

Critical.

Raw data cannot simply be transmitted.

Need understanding:

Why Encoding Exists

Problems:

Clock Recovery

DC Balance

Control Symbols

8b/10b

Maps:

8 bits

into

10 bits

Provides:

Transition Density

Running Disparity Control

Special Characters

Used heavily in Fibre Channel.

Need understanding:

Data Characters

Control Characters

No need to memorize tables yet.

MODULE 0.5.8

SERDES

Serializer/Deserializer.

Heart of modern protocols.

Input:

32-bit parallel

Output:

Serial stream

Receiver performs reverse.

Need understanding:

Serialization

Deserialization

Clock Recovery

Alignment

Without SERDES:

ARINC 818 impossible.

MODULE 0.5.9

Xilinx GT Transceivers

For FPGA engineers this is mandatory.

Most engineers initially treat GTs as magic.

Don't.

Understand architecture.

Simplified:

FPGA Logic

↓

PCS

↓

PMA

↓

Serializer

↓

Pins

Need understanding:

GTX

GTH

GTY

Families differ.

Concepts similar.

PCS

Protocol-oriented functions.

Examples:

8b/10b

Alignment

Elastic Buffer

PMA

Physical signaling layer.

Handles:

Serialization

Clock Recovery

Analog Functions

Reference Clock

Critical.

GT requires stable reference clock.

MODULE 0.5.10

Clock Recovery

Huge concept.

Question:

Receiver receives data.

How does it know when to sample?

Need:

CDR

Clock Data Recovery

Receiver extracts clock from data transitions.

This is why encoding matters.

MODULE 0.5.11

Alignment

Another hidden topic.

Receiver sees:

101011001010...

Where does word begin?

Need alignment.

Concepts:

Bit Alignment

Byte Alignment

Word Alignment

Special symbols help.

MODULE 0.5.12

Error Detection In High-Speed Links

Need understanding:

Bit Errors

Burst Errors

CRC

BER

Example:

BER = 10^{-12}

Meaning.

Must understand.

MODULE 0.5.13

Link Bring-Up

Before protocol works:

Link must work.

Bring-up sequence:

Power



Reference Clock



GT Reset



PLL Lock



CDR Lock



Alignment



Link Ready



Protocol Traffic

Many debugging sessions occur before protocol starts.

MODULE 0.5.14

Protocol Debugging Hierarchy

Critical professional skill.

When link fails:

Never start at ARINC.

Check:

Power?

Then:

Clock?

Then:

GT Lock?

Then:

Alignment?

Then:

Fibre Channel?

Then:

ARINC?

Debug layer by layer.

MODULE 0.5.15

ARINC 818 Conceptual Model

Without reading spec:

You should now see:

Video Source



Pixel Stream



ARINC Container



Fibre Channel Frame



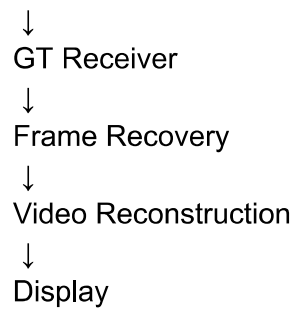
8b/10b Encoding



GT Serializer



Optical/Electrical Link



This single diagram is the bridge between everything in Phase 0 and everything we will study in ARINC 818.

PHASE 0.5 COMPLETION CHECKLIST

You should be able to explain:

Video

- Pixel
- Line
- Frame
- FPS
- Resolution
- Color depth
- Blanking

Aerospace Context

- Why ARINC 818 exists
- Aircraft display systems

Fibre Channel

- Frame structure
- SOF
- EOF
- Ordered sets
- CRC

High-Speed Links

- Differential signaling
- BER
- Jitter

- Encoding

GT Transceivers

- PCS
- PMA
- CDR
- Alignment
- Reference clock

System View

- Complete video-to-link-to-video path

Only after these concepts are understood would I start **Phase 1: ARINC 818-2 System Understanding**, where we finally open the protocol itself and study what ARINC adds on top of Fibre Channel.

No.

What we covered is approximately:

70–80% of Phase 0.5

for a typical engineer.

For someone who wants to:

- Learn ARINC 818-2 deeply
- Implement TX/RX in FPGA
- Debug hardware in a lab
- Later learn any protocol independently

there are still important gaps.

The missing parts are not basic concepts anymore.

They are the **engineering details that become painful later if not understood now.**

Remaining Phase 0.5 Gaps

I would group them into:

0.5A Video Engineering

0.5B High-Speed Link Engineering

0.5C Fibre Channel Engineering
0.5D Aerospace System Engineering
0.5E FPGA Implementation Engineering
0.5F Protocol Learning Methodology

GAP 0.5A-1

Pixel Formats

We said RGB.

Not enough.

Need understanding of:

RGB444

R = 4 bits

G = 4 bits

B = 4 bits

RGB565

Common.

R = 5

G = 6

B = 5

RGB888

Most common.

24 bits/pixel

RGB101010

Used in high-quality displays.

YUV

Critical.

Need awareness:

Y = Luma

U = Chroma

V = Chroma

Why?

Some aerospace displays use non-RGB formats.

GAP 0.5A-2

Chroma Subsampling

Need awareness of:

4:4:4

4:2:2

4:2:0

Not because ARINC requires it.

Because bandwidth calculations change dramatically.

GAP 0.5A-3

Frame Buffering

Need understanding of:

Full Frame Buffer

Entire image stored.

Advantages:

Easy processing.

Disadvantages:

Latency.

Memory.

Line Buffer

Stores one line.

Streaming Video

No full frame storage.

Pixel enters.

Pixel exits.

ARINC systems often use streaming architectures.

GAP 0.5A-4

Video Clock Domains

Need understanding:

Pixel Clock

versus

Transport Clock

These are often different.

Example:

Video = 74.25 MHz

Link = 125 MHz

Need CDC strategy.

GAP 0.5A-5

Video Test Patterns

Extremely important.

Need familiarity with:

Color Bars

Checkerboard

Ramp

Walking Pixel

Cross Hatch

These are used constantly in ARINC validation.

GAP 0.5B-1

Signal Integrity

We only introduced it.

Need deeper awareness.

Reflection

Why transmission lines behave differently at GHz speeds.

Impedance

Need conceptual understanding of:

50 Ω

100 Ω differential

Return Loss

Insertion Loss

No need for RF design expertise.

Need awareness.

GAP 0.5B-2

Jitter Types

Need distinction:

Random Jitter

Deterministic Jitter

Periodic Jitter

Professional discussions use these terms.

GAP 0.5B-3

Eye Diagrams

Need ability to interpret:

Open Eye

Good link.

Closed Eye

Bad link.

Eye height.

Eye width.

Conceptually.

GAP 0.5B-4

BER Measurement

Need understanding:

10^{-6}
 10^{-9}
 10^{-12}
 10^{-15}

Why these numbers matter.

Example:

10^{-12} BER

does not mean zero errors.

Need intuition.

GAP 0.5B-5

Link Margin

Need concept:

Current BER

versus

Margin to Failure

Important during qualification.

GAP 0.5C-1

Fibre Channel Ordered Sets

We mentioned them.

Need deeper understanding.

Ordered sets are:

Special control structures.

Not payload.

Used for:

Idle

Synchronization

Link Management

Need understanding of why control traffic exists.

GAP 0.5C-2

Fibre Channel Classes

Need awareness.

Not detailed study.

Understand:

Different service models exist.

Reason:

Many ARINC concepts inherit FC terminology.

GAP 0.5C-3

Frame Sequencing

Need awareness:

Frames belong to larger exchanges.

Important conceptually.

GAP 0.5C-4

Credit-Based Flow Control

Very important.

Need understanding of:

Receiver advertising capacity.

Why?

Prevent buffer overflow.

Even if ARINC simplifies some FC behavior, the mindset matters.

GAP 0.5D-1

Aerospace Certification Mindset

Huge gap.

Commercial FPGA projects:

Works?

Ship it.

Aerospace:

Prove it works.

Need awareness:

Verification Evidence

Traceability

Requirements

Qualification

Not certification expertise.

Just mindset.

GAP 0.5D-2

ICD Concept

Extremely important.

Many ARINC learners miss this.

ARINC standard:

Defines possibilities.

ICD:

Defines actual implementation.

In real projects:

ICD is often more important than protocol standard.

Need strong understanding of:

Protocol

≠

ICD

GAP 0.5D-3

Redundancy Concepts

Aerospace systems often use:

Primary

Secondary

Links.

Need awareness of:

Failover.

Fault tolerance.

GAP 0.5E-1

GT Clocking Architecture

Critical.

Need understanding:

Reference Clock

vs

Recovered Clock

vs

User Clock

These confuse many engineers.

GAP 0.5E-2

GT Reset Sequencing

Not just "assert reset."

Need understanding:

GTs have specific startup sequences.

Many first-time users struggle here.

GAP 0.5E-3

Elastic Buffers

Critical concept.

Need understanding:

Compensate frequency differences.

Without them:

Long-term drift causes errors.

GAP 0.5E-4

Clock Compensation

Need conceptual understanding.

Two systems are never exactly identical frequency.

Need compensation mechanisms.

GAP 0.5E-5

Protocol Analyzer Mindset

Need awareness of tools:

Oscilloscope

Logic Analyzer

ILA

Protocol Analyzer

Each sees different abstraction levels.

GAP 0.5F-1

How To Read A Protocol Standard

This may be the most important missing topic.

When handed:

600-page protocol document

Never start at page 1.

Study order:

Purpose

↓

Architecture

↓

Terminology

↓

Frame Format

↓

State Machines

↓

Timing

↓

Exceptions

This skill transfers to every future protocol.

GAP 0.5F-2

Protocol Reverse Engineering

Need ability to ask:

Why is this field present?

instead of:

What is this field?

This dramatically speeds learning.

GAP 0.5F-3

Layer Isolation

Critical debugging skill.

When protocol fails:

Separate:

Application

from

Protocol

from

Link

from

Physical Layer

Never debug all layers simultaneously.

Final Hidden Requirement

There is one thing almost every protocol expert develops.

Not knowledge.

A habit.

For every protocol element ask:

What problem is being solved?

Example:

CRC

Question:

What failure occurs if removed?

FIFO

Question:

What failure occurs if removed?

8b/10b

Question:

What failure occurs if removed?

Ordered Set

Question:

What failure occurs if removed?

This habit is what allows engineers to learn unfamiliar protocols quickly.

True Phase 0.5 Exit Criteria

Before opening the ARINC 818-2 specification, you should be able to draw from memory:

Video Source



Pixel Stream



Frame Buffer/FIFO



ARINC Packaging



Fibre Channel Frame



8b/10b Encoder



GT Transmitter



Differential Link



GT Receiver



Clock Recovery



Frame Recovery



Video Reconstruction

↓
Display

And for every block answer:

1. What goes in?
2. What comes out?
3. What clock is used?
4. What buffer exists?
5. What errors can occur?
6. How are errors detected?
7. How would I debug it?

When you can do that, Phase 0.5 is effectively complete for an FPGA engineer preparing to study ARINC 818-2 professionally.

At that point, the next step is no longer prerequisites. It is **Phase 1: Understanding ARINC 818-2 as a complete system**, starting from the avionics use case and the relationship between ARINC 818, Fibre Channel, video sources, displays, and the ICD concept.

No.

But now we are reaching the point where the remaining gaps are not "topics." They are **professional mental models** that separate someone who can *study* ARINC 818 from someone who can *own* an ARINC 818 design.

If I were training an FPGA engineer for aerospace protocol work, there is one final layer of Phase 0.5 that most books, courses, and even senior engineers never explicitly teach.

Let's call it:

Phase 0.5 Final Layer

Protocol Engineering Mindset

This is the layer that makes future protocols easier.

Gap 0.5-Final-1

The Protocol Stack Mental Model

Most people learn:

ARINC 818

Wrong approach.

Engineers see:

Application



Protocol



Transport



Encoding



Serialization



Physical Signaling

For ARINC 818:

Video



ARINC 818



Fibre Channel Framing



8b/10b



GT SERDES



Differential Pair

When debugging:

Never ask:

ARINC broken?

Ask:

Which layer is broken?

This mindset transfers to:

- Ethernet
 - PCIe
 - JESD204
 - SpaceWire
 - Aurora
 - USB
 - SATA
-

Gap 0.5-Final-2

Data Journey Thinking

For every protocol, track one piece of data.

Example:

Pixel

Follow it:

Pixel

↓

Word

↓

Frame

↓

Encoded Symbol

↓

Serial Bits

↓

Voltage

↓

Serial Bits

↓

Decoded Symbol

↓

Frame

↓

Pixel

If you cannot follow one piece of data end-to-end, you do not yet understand the protocol.

Gap 0.5-Final-3

Time Journey Thinking

Most engineers follow data.

Few follow time.

Example:

A pixel arrives.

Ask:

When?

Then:

How long in FIFO?

Then:

How long in serializer?

Then:

How long on wire?

Then:

How long in receiver?

Then:

When displayed?

This is latency analysis.

Very important in avionics.

Gap 0.5-Final-4

Clock Journey Thinking

For every signal ask:

Which clock owns it?

Example:

Video Clock

↓

FIFO

↓

Transport Clock

↓

GT Clock

↓

Recovered Clock

Many protocol bugs are actually clock bugs.

Gap 0.5-Final-5

Buffer Journey Thinking

For every buffer ask:

Why does it exist?

What happens if removed?

Example:

Video FIFO

Purpose?

Rate matching.

Without it?

Overflow.

Underflow.

Frame corruption.

Gap 0.5-Final-6

Failure Propagation Thinking

Professional engineers think:

Failure

↓

Effect

↓

Detection

↓

Recovery

Example:

Bit Error

↓

CRC Error

↓

Frame Drop

↓

Image Artifact

Understand failure chains.

Gap 0.5-Final-7

Recovery Thinking

Every protocol has:

Normal Path

and

Recovery Path

Questions:

What if link disappears?

What if clock lost?

What if frame corrupted?

What if FIFO overflows?

Often recovery logic is larger than normal logic.

Gap 0.5-Final-8

Bandwidth Budget Thinking

Engineers don't estimate.

They budget.

Example:

1080p60 RGB
≈ 3 Gbps raw

Now ask:

Encoding overhead?

Protocol overhead?

Margin?

Actual link requirement is larger.

Gap 0.5-Final-9

Latency Budget Thinking

Example:

Requirement:

< 1 ms

Now allocate:

Capture
100 μ s

FIFO
200 μ s

Transport
100 μ s

Display
300 μ s

Margin
300 μ s

This mindset is critical in aerospace.

Gap 0.5-Final-10

Resource Budget Thinking

Every feature costs:

LUTs

FFs

BRAMs

Latency

Verification Effort

Need awareness.

Gap 0.5-Final-11

Specification Reading Discipline

When reading any protocol:

Never start memorizing fields.

Study in this order:

Purpose

Why does protocol exist?

Architecture

Major blocks.

Data Unit

Frame.

Packet.

Container.

Word.

State Machines

Behavior.

Timing

When things happen.

Exceptions

What happens when things go wrong.

Only then read field definitions.

Gap 0.5-Final-12

Implementation Decomposition

When handed a protocol:

Immediately decompose into blocks.

Example:

RX

Deserializer

↓

Decoder

↓

Frame Detector

↓

Parser

↓

CRC Checker

↓

Buffer



Output

This skill is reusable across protocols.

Gap 0.5-Final-13

Verification Decomposition

Every protocol block should have:

Stimulus

Checker

Coverage

Error Injection

Mindset:

If it cannot be verified, it is not finished.

Gap 0.5-Final-14

Lab Debug Decomposition

Debug hierarchy:

Power



Clock



Reset



GT



Encoding

↓
Framing
↓
Protocol
↓
Application

Never skip layers.

Gap 0.5-Final-15

ICD-Centric Thinking

This is probably the most important ARINC-specific lesson.

New engineers think:

I need ARINC 818.

Reality:

You need:

ARINC 818

+

Program ICD

The ICD defines:

- Resolution
- Frame rate
- Container structure
- Link speed
- Object definitions
- Timing

Real projects are built to ICDs, not to standards alone.

The Final Readiness Test

Before Phase 1, I would expect you to be able to answer:

Video

- How many bits per second does a video stream generate?
- What changes bandwidth?

Fibre Channel

- Why use frames?
- Why use ordered sets?
- Why use CRC?

High-Speed Links

- Why use differential signaling?
- Why use encoding?
- Why use CDR?

GT Transceivers

- What does PCS do?
- What does PMA do?

Protocol Engineering

- Where is buffering required?
- Where is clock crossing required?
- What are possible failure modes?

Aerospace

- Why is determinism important?
- Why is an ICD required?

System Thinking

- Trace a pixel from source to display.
 - Trace a clock through the system.
 - Trace an error through the system.
-

Is Phase 0.5 Now Complete?

For practical engineering purposes:

Yes.

Not because there is literally nothing left to learn—there always is—but because the remaining knowledge now belongs naturally inside:

- Phase 1 ARINC 818 System Understanding
- Phase 2 ARINC 818 Architecture
- Phase 3 ARINC 818 Containers and Objects
- Phase 4 Fibre Channel Details
- Phase 5 FPGA Implementation
- Phase 6 Verification
- Phase 7 Lab Bring-Up

At this point, additional Phase 0.5 material would mostly be leaking Phase 1 content backward.

So for an FPGA engineer preparing to implement and debug ARINC 818-2 professionally, Phase 0 and Phase 0.5 together form a sufficiently complete foundation to begin studying ARINC 818 itself.